

Your snake, live

about 30 minutes

Right now your snake game is a file on your laptop. In half an hour it's a web address, and every time you push a change the address updates itself. Nobody uploads anything.

Nobody teaches you GitHub today either. Claude does, while it does the work, pitched at whatever you already know. The person next to you gets a different lesson.

You need the snake-code folder from Lab 4 of Four snakes. If the folder is gone, make a new empty one called snake-code, open it in the Code screen and type:

```
Make a snake game in one HTML file called index.html. No framework, nothing to install.
```

1. OPEN THE FOLDER AND SIGN IN

In the desktop app, open the Code screen. snake-code is listed in the sidebar from this morning's lab; click the + next to it to start a new session in that folder.

Claude will do the GitHub work through GitHub's command-line tool, gh. It isn't on your laptop yet. Have Claude put it there and sign you in. Type:

```
Check whether the GitHub CLI, gh, is installed. If it isn't, install it. Then sign me in to GitHub with it. Tell me what you're doing before each step.
```

Signing in opens a browser window: GitHub shows you a short code, you type it in and approve. Use the GitHub account you made before class. When Claude says you're signed in, you're ready.

2. PICK YOUR LEVEL

Pick the one sentence that's true and keep it for the next step:

- I've never used GitHub. Assume I know nothing. The first time you use a word like repo, commit, push, branch, Pages, Actions or workflow, stop and explain it in plain words, with a short example, before you go on.
- I've used GitHub a bit. I know what a repo, a commit and a push are. Explain Pages, Actions and the workflow file properly as we get to them.
- I use GitHub daily. Skip the basics and only explain what I'd not have seen before.

3. PUT IT ON THE INTERNET

Type this, with your sentence in the first line:

```
[your level sentence]
```

```
Put this game on the internet, and teach me as you go. Before each step,
tell me what you're about to do and why, at my level, then do it. Don't
skimp on the teaching; it's the point. If the game isn't called index.html,
rename it first. Add a
.gitignore file that lists .env. Make a public GitHub repo called snake and
push this folder to it. Turn on GitHub Pages with GitHub Actions as the
source. Then add a workflow that publishes the folder every time I push to
main, and push that too. When the site is live, give me both addresses.
```

Claude explains each GitHub step before it does it. Read along as it goes; that's your lesson. If it stops to ask you something, read what's above the question first.

If it asks you something you don't know the answer to, say so and tell it to pick.

4. GO LOOK

Open the repo's address and click the Actions tab. There's a run there with a green tick. That run is a machine at GitHub that woke up when your push landed, followed the workflow file, and published your game. It took somewhere between fifteen seconds and a minute and a half, and it's already over.

While you're in the repo, click `.gitignore`. One line: `.env`. That file is the list of what never gets pushed. Nothing in this folder needs it yet. The day your project has a key in it, this is what keeps the key off the internet.

Now open the game:

```
https://[your GitHub username].github.io/snake/
```

Play it. Then open it on your phone.

If the run is red, click into it, read the message, and tell Claude what it says. Describe what you saw; don't guess the cause.

A green run with a yellow warning about Node.js is still a green run. Leave it.

5. CHANGE IT AND WATCH

Keep the Actions tab where you can see it. This time you watch the run happen. In the Code screen:

```
Make the snake blue. Commit and push.
```

A new run appears with a yellow dot. When it goes green, reload your game. If the snake is still the old colour, your browser is showing you its saved copy. Hold Shift and click reload.

6. ASK THE ONE THING YOU DIDN'T GET

Something went past you in step 3. Ask about it now, in the same session:

```
Go back over what you did. The part I didn't follow was [whatever it was].  
Explain just that, at my level.
```

YOU'RE DONE WHEN

- Your URL plays your game.
- You've changed the game once and watched the run that put the change live.
- You asked Claude about the one part you didn't follow.